

CoX-MoE: Coalesced Expert Execution for High-Throughput MoE Inference with AMX-Enabled CPU–GPU Co-Execution

Muyoung Son*
KAIST
Daejeon, Republic of Korea
kkt1690@kaist.ac.kr

Yi Chen*
KAIST
Daejeon, Republic of Korea
chenyi@kaist.ac.kr

Seungjae Yoo
KAIST
Daejeon, Republic of Korea
goldenyoo@kaist.ac.kr

Soongyu Choi
KAIST
Daejeon, Republic of Korea
soongyu1291@kaist.ac.kr

Joo-Young Kim†
KAIST
Daejeon, Republic of Korea
jooyoung1203@kaist.ac.kr

Abstract

The Mixture-of-Experts (MoE) architecture improves computational efficiency via sparse expert activation, but throughput-oriented inference faces substantial GPU memory pressure due to a significant parameter size and intermediate data. Prior works attempt to mitigate this using expert offloading with micro-batching or by offloading computation to the CPU. However, the fragmented workload resulting from micro-batching degrades operational intensity, causing expert execution to become memory-bound. Meanwhile, CPU offloading is constrained by slow PCIe transfers and its limited applicability to attention computation in the decode stage. Consequently, these inefficiencies prevent effective system utilization, severely restricting the end-to-end throughput of MoE inference.

To address these challenges, this paper proposes CoX-MoE, an Advanced Matrix Extensions (AMX)-enabled CPU–GPU collaborative system that comprehensively optimizes MoE inference by combining coalesced expert execution with strategic workload orchestration for higher throughput. CoX-MoE introduces (i) a coalescing-aware orchestration policy to jointly optimize resource allocation by adopting ordinary batch, instead of micro-batch, for expert computation and selective attention offloading, and (ii) a static expert-aware stratification scheme that pre-assigns frequently activated experts to the GPU, mitigating PCIe transfer overhead and balancing workload for the CPU and GPU during inference. Compared to state-of-the-art frameworks, CoX-MoE delivers significant gains, achieving up to 7.1× and 2.4× higher throughput than FlexGen and MoE-Lightning, respectively.

CCS Concepts

• **Computer systems organization** → **Heterogeneous (hybrid) systems**; • **Computing methodologies** → **Natural language processing**.

*Both authors contributed equally to this research.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
DAC '26, Long Beach, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2254-7/2026/07

<https://doi.org/10.1145/3770743.3804296>

Keywords

MoE Inference, Offloading, Workload Orchestration, Heterogeneous Computing, AMX

ACM Reference Format:

Muyoung Son, Yi Chen, Seungjae Yoo, Soongyu Choi, and Joo-Young Kim. 2026. CoX-MoE: Coalesced Expert Execution for High-Throughput MoE Inference with AMX-Enabled CPU–GPU Co-Execution. In *63rd ACM/IEEE Design Automation Conference (DAC '26)*, July 26–29, 2026, Long Beach, CA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3770743.3804296>

1 Introduction

Mixture-of-Experts (MoE) has emerged as a promising architecture for scaling model capacity without proportionally increasing computation cost [16, 28]. Unlike dense large language models (LLMs) [3, 19], which activate all parameters, MoE activates only a subset of experts for each token, reducing active parameters during inference while maintaining high performance. As a result, MoEs are considered more computationally efficient than dense LLMs.

Despite its advantage, the MoE architecture poses a severe GPU memory (VRAM) challenge. MoE models typically have larger parameter sizes than dense LLMs, despite having similar active parameters. Moreover, throughput-oriented workloads, such as offline benchmarking [14], large-scale data processing [18], or synthetic data generation [17], further inflate VRAM usage due to large intermediate activations. For example, Mixtral-8x22B [11] requires about 282 GB for BF16 weights, and a 64-batch, 4096-token workload adds roughly 72 GB of intermediate data, bringing the total to around 350 GB. Such footprints require multiple high-end GPUs (e.g., roughly five 80 GB H100s [6], costing up to \$200,000), which is impractical for most deployments and motivates MoE inference systems that focus on single-GPU, throughput-intensive workloads.

To operate high-throughput inference within a limited VRAM capacity, prior works [2, 22] combine memory offloading with micro-batching. The memory offloading strategy involves offloading model weights or activation data into host memory and swapping them between VRAM and host memory via PCIe, enabling inference for models that exceed VRAM capacity. For MoE models, memory offloading takes the form of expert offloading [11, 23, 24], where non-expert and a subset of expert weights remain resident in VRAM, while other expert weights are fetched from host memory via on-demand [23] or prefetch [9, 23, 26] mechanisms. To maximize

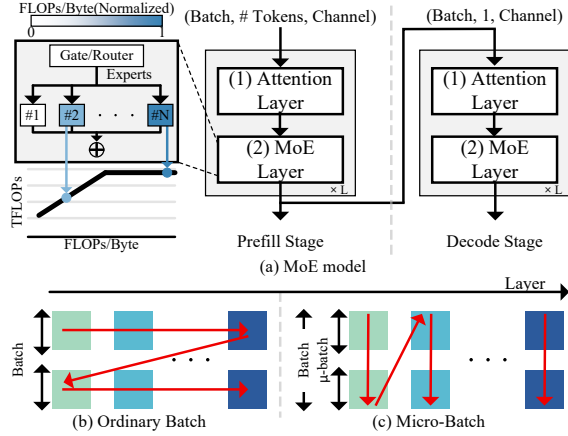


Figure 1: (a) Architecture of Mixture-of-Experts along with the inference flow. (b), (c) Inference flow for each strategy.

the efficiency of memory offloading, large input batches are divided into multiple micro-batches, which are executed sequentially on the GPU, since activations for the entire batch often exceed VRAM capacity. As shown in Fig. 1(c), this design promotes weight reuse across micro-batches and alleviates memory pressure. In alternative approaches, other systems [4, 12, 27] offload expert operations to the CPU to reduce PCIe transfer, which is one of the primary bottlenecks, and examine the activated experts at each step to decide whether each should be executed on the GPU or the CPU.

However, these systems face two limitations for MoE inference. First, micro-batching fragments each expert’s workload, lowering its operational intensity and causing expert computations to become memory-bound, where latency is dominated by VRAM access. This bottleneck is then exacerbated by repeatedly loading the same expert weights from VRAM in every micro-batch, resulting in increased per-layer latency and reduced overall throughput.

Second, existing CPU-assist solutions primarily rely on Intel Advanced Vector Extensions (AVX) [10] and target offloading attention-related GEMV operations in the decode stage, while leaving the GEMM-intensive prefill stage largely unexploited. The reason is that the limited per-core matrix multiplication throughput of AVX is insufficient to enable meaningful compute offloading. Furthermore, attempting to offload expert computation is hampered by fundamental hardware limitations: the slow PCIe connection, and the significant performance gap between the CPU and GPU may lead to workload imbalance, thereby limiting the overall performance.

To overcome these challenges, we propose CoX-MoE, an Advanced Matrix Extensions (AMX)-enabled CPU–GPU collaborative system that creates an optimization framework centering on (i) coalescing-aware orchestration policy and (ii) expert-aware stratification to maximize MoE inference throughput. In summary, we make the following contributions:

- We analyze the workload of MoE inference, identifying key performance bottlenecks and optimization opportunities by leveraging modern CPU instructions with Intel AMX.
- We propose and implement a coalescing-aware orchestration policy implemented within an efficient AMX-enabled CPU–GPU framework, which jointly optimizes compute/expert allocation, while enforcing ordinary batch for expert computation instead

of micro-batch, and utilizing an attention offloading strategy that frees up VRAM by assigning partial or complete attention operations to the CPU or GPU.

- We design an expert-aware stratification scheme to identify frequently activated experts, based on batch clustering, sampling and probing, enabling ahead-of-time expert placement that maximizes resource utilization and balances the workload.
- Overall, CoX-MoE achieves a 1.7–2.4× throughput improvement over a state-of-the-art (SOTA) offloading scheme.

2 Background

2.1 MoE Models and Inference Workloads

As shown in Fig. 1(a), MoE models share the same two-phase inference with dense LLMs [8]. The prefill stage processes the entire batch of input sequences at once. In contrast, the decode stage generates tokens autoregressively, one token at a time. While dense LLMs activate all parameters for every input token, MoE models employ a routing function to dynamically route each token to a sparse subset of experts, for example, selecting the top- k from a much larger pool of N total experts [5]. This sparse and uneven token-to-expert routing results in varying workloads across individual experts, leading to substantial variation in operational intensity.

2.2 Advanced Matrix Extensions

To accelerate CPU-side inference for ML workloads, Intel introduced AMX, an on-chip matrix multiplication accelerator with dedicated ISA support, starting with the 4th generation Xeon (2022) [1, 13]. The AMX architecture comprises two key components: (1) a 2D array of registers (Tile) and (2) a Tile Matrix-Multiply (TMUL) unit, both designed to operate efficiently on INT8 and BF16 data formats. To deliver high inference speed, the CPU issues dedicated AMX instructions which execute on multi-cycle AMX units.

Compared to AVX-512 [10], which delivers approximately 18 TFLOPs of peak BF16 performance per socket, Intel AMX on Sapphire Rapids increases per-socket matrix multiplication throughput to approximately 144 TFLOPs, about an order of magnitude higher. This substantial improvement narrows the gap to a high-end GPU, such as the RTX 6000 Ada, at approximately 364 TFLOPs. This elevated CPU-side throughput with AMX makes CPU–GPU co-execution a practically effective strategy for compute offloading.

2.3 Offloading and Micro-Batch Strategy

Traditional inference systems face significant VRAM pressure when model size exceeds memory or when processing large batches. To address this, unlike normal batch inference shown in Fig. 1(b), prior works [2, 22] offload model weights and activations to host memory or SSDs and process each batch as a sequence of micro-batches on the GPU, as illustrated in Fig. 1(c). By executing these micro-batches sequentially, the system enables weight reuse: the expert weights are loaded into VRAM once and reused across all micro-batches, significantly reducing repeated PCIe transfers. The micro-batch size is optimized to balance between PCIe transfer time and on-GPU efficiency. For MoE models, a specialized mechanism known as expert offloading is used [23, 24, 26]. Non-expert and a subset of expert weights remain resident on the GPU, while the other experts are stored in host memory or on an SSD. When activated, these offloaded experts are either fetched to the GPU (via prefetch or on-demand loading), or computed directly on the CPU [4, 12, 27].

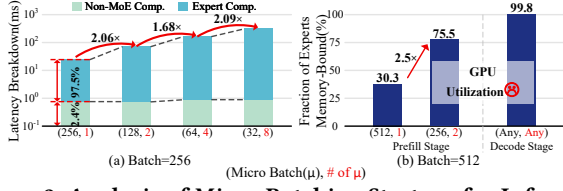


Figure 2: Analysis of Micro-Batching Strategy for Inference.

Furthermore, in the decode stage, because the KV cache grows dynamically and makes attention heavily memory-bound, repeatedly transferring it over the slow PCIe bus is inefficient. Therefore, prior works [2, 22] also fully offload decode-stage attention to the CPU to operate directly on KV data in host memory.

However, these approaches remain limited by the low operational intensity of expert execution under micro-batching and the insufficient CPU throughput in compute offloading. Furthermore, existing works suffer from severe workload imbalance when offloading experts and have a narrow focus on decode-stage attention.

3 Motivation

In this section, we use NVIDIA Nsight Systems/Compute [20] to profile the Qwen3-30B-A3B [25] on an NVIDIA RTX 6000 Ada Generation GPU with an Intel Xeon Platinum 8452Y CPU, identifying performance bottlenecks and characterizing MoE batch inference behavior. We evaluate batch sizes of 128, 256, and 512 with a prompt length of 512, following prior work [21, 27].

3.1 Low Expert Arithmetic Intensity

As shown in Fig. 2(a), E2E latency increases with the number of micro-batches, while non-expert latency remains nearly constant. In contrast, expert computation, which accounts for over 97.5% of total latency, is highly sensitive to micro-batching: halving the micro-batch size nearly doubles expert latency. This is because sparse routing gives each expert fewer tokens than non-expert operations, and smaller micro-batches further reduce per-expert inputs, lowering operational intensity and pushing expert execution into the memory-bound regime. Accordingly, Fig. 2(b) shows that more experts become memory-bound as micro-batch size decreases.

Once an expert becomes memory-bound, its execution time is dominated by repeated fetches of expert weights from VRAM per micro-batch, which in turn increases the total VRAM access cost and directly leads to a near-linear increase in expert latency. In contrast, non-expert computation remains stable, as it has not yet entered the memory-bound regime under the evaluated micro-batch sizes. Consequently, prior work that relies on micro-batching dramatically exacerbates the primary latency bottleneck by inflating expert computation time, severely limiting overall throughput.

Opportunity 1: Coalesced Expert Execution with AMX-Assisted Co-Execution. The severe penalty of micro-batching reveals a key opportunity: execute experts over the entire batch instead of splitting them into micro-batches. This coalesced execution increases operational intensity, but also concentrates computation on the GPU. Meanwhile, as discussed in Section 2.2, AMX provides high enough performance for the CPU to offload a meaningful portion of expert computation. These observations motivate AMX-enabled CPU-GPU co-execution for higher throughput.

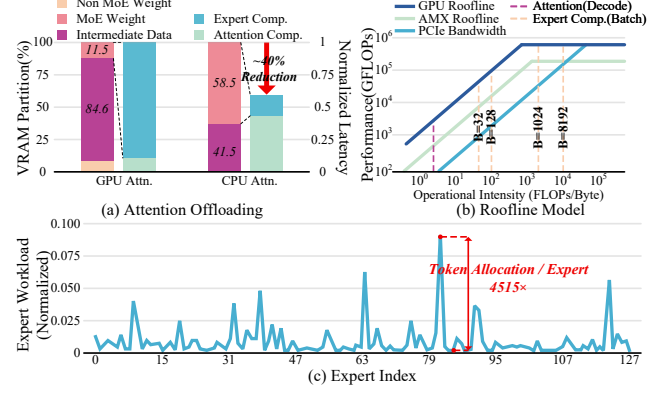


Figure 3: (a) VRAM partitioning and Latency analysis via Attention Offloading. (b) GPU and AMX Roofline Model with PCIe. (c) Normalized expert workload distribution at Layer 1 on the BIG-Bench Hard (bbh) benchmark.

3.2 Intermediate Data-driven VRAM Pressure

Fig. 3(a) illustrates the breakdown of VRAM consumption during MoE inference, highlighting the substantial memory pressure imposed by attention operations in the prefill stage. In the baseline “GPU Attn”, which executes attention operations on the GPU, the vast majority of VRAM (84.6%) is consumed by intermediate data generated during computation. It severely constrains the VRAM available for expert weights, limiting them to only 11.5% of the total partition. Consequently, most experts must be offloaded from the GPU, forcing either frequent data transfers over the PCIe bus or imposing a heavy computational load on the CPU. This memory constraint severely exacerbates the expert computation bottleneck.

Opportunity 2: Optimal VRAM Allocation via Attention Offloading. We analyze a “CPU Attn” that offloads the attention computation from the GPU to the CPU using AMX, allowing more expert weights to be stored in VRAM, expanding to 58.5%. This strategy introduces a clear trade-off: the attention computation latency increases because data is moved into the CPU. However, keeping more experts resident in VRAM dramatically reduces expert computation latency. As shown in Fig. 3(a), this reduction exceeds the additional attention delay, with AMX providing sufficient CPU-side GEMM capability, resulting in a 40% reduction in total inference latency. This finding implies that for large-batch MoE model inference, it is more effective to strategically offload attention computation, which generates the voluminous intermediate data, rather than offloading expert weights.

3.3 PCIe Bottleneck and Workload Imbalance

As shown in Fig. 3(b), offloading experts and their computations introduces two critical performance bottlenecks. The first is the PCIe bandwidth bottleneck, where the PCIe transfer rate (≈ 32 GB/s) for fetching experts to the GPU is $10\times$ slower than the CPU’s DDR5 memory bandwidth (≈ 300 GB/s). This significant difference ensures that dynamically transferring experts over the PCIe bus results in significant I/O overhead. The second is the asymmetry in computational power. As discussed in Section 3.1, Intel AMX significantly boosts CPU throughput, making co-execution feasible. However, a performance gap with the GPU still remains. This means

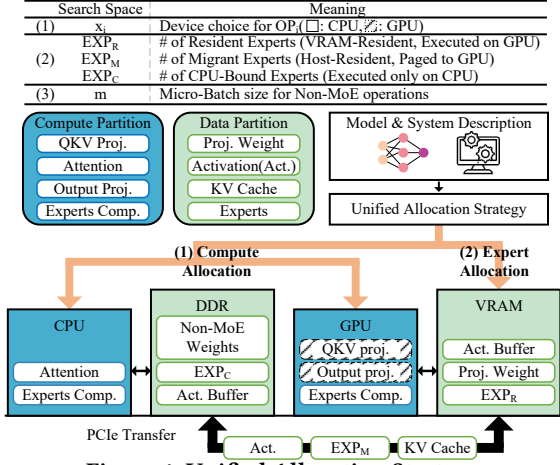


Figure 4: Unified Allocation Strategy.

Table 1: Per-micro-batch (m) data size and FLOPs of decoder operations (BF16).

Idx(i)	Operation(OP_i)	D_X (Byte)	D_Y (Byte)	C (FLOP)
0	QKV Projection	$2mLd_h$	$6d_h^2$	$6mLd_h^2$
1	Attention(Prefill)	$2mLd_h$	$4mLd_h$	$4mL^2d_h$
	Attention(Decode)		$4mL_{kv}d_h$	$4mL_{kv}d_h$
2	Out. Projection	$2mLd_h$	$2d_h^2$	$2mLd_h^2$
3	Per-Expert Computation	$\frac{2BLd_h}{E}$	$6d_h d_e$	$\frac{6BLd_h d_e}{E}$

B : Batch size, E : Number of activated experts, d_h : model (hidden) dim, d_e : expert dim, L : sequence length (equals 1 in the decode stage), L_{kv} : decode stage KV length.

that simply offloading experts without considering their workload intensity can lead to a workload imbalance, shifting the bottleneck from the GPU to the CPU. Therefore, an effective co-execution policy must carefully balance this asymmetry by assigning the appropriate workload to each device.

Opportunity 3: Static Allocation of Skewed Experts into VRAM. Our empirical analysis reveals that MoE architectures are characterized by sparse and uneven activated experts across all layers, as exemplified by Fig. 3(c): usage frequency varies drastically among experts, indicating that not all experts contribute equally to the computational workload. This skewed expert activation pattern suggests an opportunity to statically prioritize frequently used experts in VRAM, which is crucial for minimizing PCIe overhead and balancing the overall system throughput.

4 CoX-MoE Design

4.1 Coalescing-Aware Orchestration Policy: Micro-Batch is not all you need.

CoX-MoE establishes an optimal offloading policy based on computational characteristics to minimize e2e latency during the MoE model inference in a CPU-GPU system. First, *Unified Allocation Strategy*, as shown in Fig. 4, jointly optimizes two coupled decisions: (1) compute allocation for operations and (2) expert allocation to VRAM. Second, with *Strategy-aware Micro-batch Determination*, CoX-MoE selects the micro-batch size m for non-MoE operations (implying the number of micro-batches $M = \lceil B/m \rceil$), while fixing expert computation to use a coalesced batch of size B . These two-step decisions are established to minimize per-layer latency (T_l)

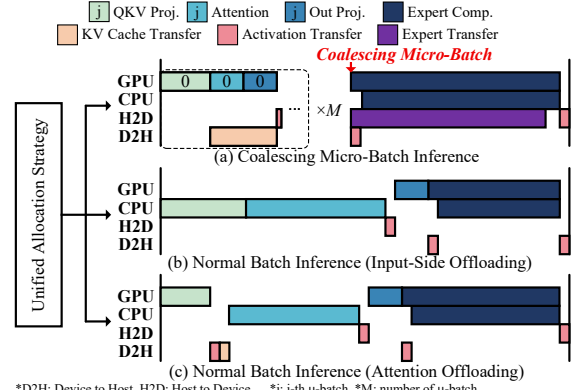


Figure 5: Example of timing diagram for a single layer of Qwen3-30B-A3B. Note that block sizes do not represent actual latency.

to achieve the optimum (T_{opt}), thereby reducing the total latency (T_{tot}) for the entire model.

$$T_{tot} = \sum_{l=1}^N T_l \geq N \times \min(T_l) = N \times T_{opt} \quad (1)$$

Here, N is the total number of decoder layers, and T_l represents the latency of the l -th decoder layer. As shown in Table 1, CoX-MoE partitions a decoder layer into four operation units, OP_0 (QKV projection), OP_1 (attention), OP_2 (output projection), and OP_3 (FFN of experts), which represent Q/K/V formation via linear maps, attention with KV-cache access, projection to the model hidden size, and gated FFN execution per expert, respectively. In addition, Table 1 summarizes the data size of the first and second matrix operands (D_{X_i} , D_{Y_i}) and FLOP(C_i) in each operation, with quantities expressed per m or per B as noted.

4.1.1 Unified Allocation Strategy. As shown in Fig. 4, CoX-MoE jointly optimizes two coupled decisions. First, as mentioned in Section 3.2, it assigns each of the three non-MoE operations exclusively to either the CPU or the GPU. Second, as mentioned in Section 3.1, it prioritizes coalesced expert execution—processing the entire batch B as a single unit—while enabling collaborative execution between the CPU and GPU.

The optimal single-layer latency (T_{opt}) aims to minimize the sum of the latencies of operational units.

$$T_{opt} = \min \left(\sum_{i=0}^3 (T(OP_i)) \right) \quad (2)$$

The latency of each operation ($T(OP_i)$, where $i \in \{0, 1, 2, 3\}$) is modeled as the sum of its loading time the two operand matrices in a given operation (T_{load}), the computation time (T_{comp}) and KV cache store time (T_{store}).

$$T(OP_i) = T_{load}(OP_i) + T_{comp}(OP_i) + T_{store}(OP_i) \quad (3)$$

T_{load} accounts for PCIe transfers of activations when the execution device changes between consecutive stages.

$$T_{load}(OP_i) = \begin{cases} MD_{X_i}/BW_{PCIe}, & \text{if } x_i \neq x_{i-1}, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

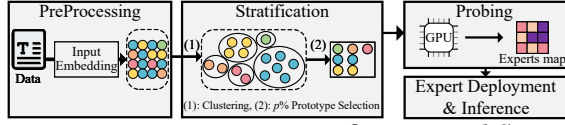


Figure 6: Expert-Aware Stratification Workflow.

T_{comp} follows a roofline model based on the hardware configuration of each device. The latency is determined by the bottleneck resource, which is either the memory bandwidth (BW) or the computational performance (TF).

$$T_{comp}(OP_i) = M \max((D_{X_i} + D_{Y_i})/BW_{DEV}, C_i/TF_{DEV}) \quad (5)$$

Finally, T_{store} is

$$T_{store}(OP_i) = \begin{cases} MD_{KV}/BW_{PCIe}, & \text{if } i = 1, x_0 = 1, x_1 = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

where D_{KV} is the per micro-batch KV cache size generated from OP_0 . The transfer via PCIe is incurred precisely when attention runs on the CPU ($x_1 = 0$) while QKV runs on the GPU ($x_0 = 1$).

With an allocation exclusively to OP_0 - OP_2 , the optimization focus shifts to OP_3 . Under VRAM capacity and PCIe bandwidth constraints, the number of experts to process on each device should be determined. We classify experts into three groups: EXP_R , EXP_M , and EXP_C , as depicted at Fig. 4. As in Eq. (3), we now specialize the per-operation latency model to the expert stage OP_3 by assuming no micro-batch ($m_{exp} = B$) and parallel CPU-GPU execution.

$$T_{load}(OP_3) = (D_{X_3} + EXP_M D_{Y_3})/BW_{PCIe} \quad (7)$$

$$T_{comp}(OP_3) = \max(\text{Latency}_{GPU}, \text{Latency}_{CPU}) \quad (8)$$

$$\text{Latency}_{DEV} = \max((D_{X_3} + EXP_{DEV} D_{Y_3})/BW_{DEV}, C_3/TF_{DEV}) \quad (9)$$

Where $DEV \in \{\text{CPU}, \text{GPU}\}$, with $EXP_{DEV} \in \{EXP_R + EXP_M, EXP_C\}$. This ties (EXP_R, EXP_M, EXP_C) directly to $T(OP_3)$ and thus to T_{opt} . As shown in Table 1, OP_3 uses an ordinary batch B for D_X , effectively setting the micro-batch size for this stage to B .

4.1.2 Strategy-Aware Micro-Batch Determination. Given the above strategy, CoX-MoE determines the optimal m for OP_0 - OP_2 that satisfies Eq. 1. under the VRAM budget. The budget explicitly accounts for EXP_R weights, computes allocation-induced non-MoE weights, intermediate data buffers (for the attention/expert computation), and temporary working space for kernels. CoX-MoE then searches the feasible space of $(x_0, x_1, x_2, EXP_R, EXP_M, EXP_C, m)$ that satisfies the VRAM/PCIe constraints and selects the configuration that yields the smallest T_{tot} , thereby maximizing throughput.

4.2 Performance Optimization

Once the two-step decisions are determined, the execution strategy is defined. Fig. 5 illustrates a timing diagram across prefill and decode with two primary strategies. First, in Fig. 5(a), the OP_0 - OP_2 are performed in micro-batch units, but the expert computation is performed after coalescing these micro-batches. The key optimization is overlapping data movement between the CPU and GPU with computation to remove idle gaps. The second, as exemplified by Fig. 5(b) and (c), the system adopts a normal batch inference for the entire layer. Since PCIe transfer latency tends to dominate computation, especially during the decode stage, this strategy prioritizes minimizing the transfer of expert weights between CPU and GPU.

Table 2: Evaluation Systems

Host(CPU) Platform

Intel® Xeon® Platinum 8452Y, 36 cores
8 DDR5-4800 channels, 512 GB DRAM

System Configurations

(I) CPU + NVIDIA 6000 Ada Generation Graphics Card, 48 GB, PCIe 4.0
(II) CPU + NVIDIA A100 Graphics Card, 80 GB, PCIe 4.0
(III) CPU + NVIDIA H100 Graphics Card, 80 GB, PCIe 5.0

4.3 Expert-Aware Stratification

As mentioned in Section 3.3, CoX-MoE introduces Expert-Aware Stratification (EAS), a lightweight data-driven pre-analysis framework that selects which experts should be statically preloaded into VRAM before inference, thereby minimizing dynamic PCIe transfer overhead and workload imbalance between GPU and CPU. EAS is particularly advantageous in batch inference scenarios, such as throughput-oriented benchmarking, where the entire inference workload is available a priori. In these scenarios, profiling the entire dataset is prohibitively expensive. Thus, the key challenge is to identify a representative subset of data that faithfully reflects the global expert activation pattern.

As shown in Fig. 6, EAS operates in three steps. First, in the pre-processing stage, it generates input embeddings for the entire dataset, capturing the latent semantic distribution of inputs. Second, in the stratification stage, the embeddings are clustered to form strata that group semantically similar samples. The representative prototypes are then proportionally selected per cluster, ensuring stratified coverage of the dataset. Finally, inference is performed only on these selected prototypes during a prefill-only probing phase, from which the global expert activation map is approximated. The approximated expert activation map guides static expert deployment, allowing frequently activated experts to be initialized on the GPU while keeping low-usage experts offloaded to the CPU.

5 System Implementation

To enable the efficient execution of MoE models on AMX-enabled CPU-GPU systems, we extend the Intel Extension for PyTorch (IPEX), which lacks native NVIDIA support, to facilitate interoperability with NVIDIA GPUs. Moreover, we implement fine-grained, globally shared CUDA streams that allow parallel CPU-GPU execution and VRAM buffer sharing. This design maximizes throughput by overlapping PCIe data transfers with computation, thus ensuring optimal system resource utilization.

6 Experimental Results

6.1 Experimental Setup

Platforms. As shown in Table 2, we evaluate CoX-MoE on a 36-core Intel® Xeon® Platinum 8452Y with AMX [1] support paired with A100 [7], H100 [6], and RTX 6000 Ada Generation GPUs.

Models. We evaluate our system using three representative MoE models with distinct characteristics: Mixtral-8x7B-Instruct (Mixtral) [11], DeepSeek-V2-Lite (DeepSeek) [15], and Qwen3-30B-A3B (Qwen3) [25]. These models differ in the number and size of experts, as well as their architectural configurations. Mixtral employs fewer, larger experts, in contrast to the numerous smaller experts in Qwen3 and DeepSeek.

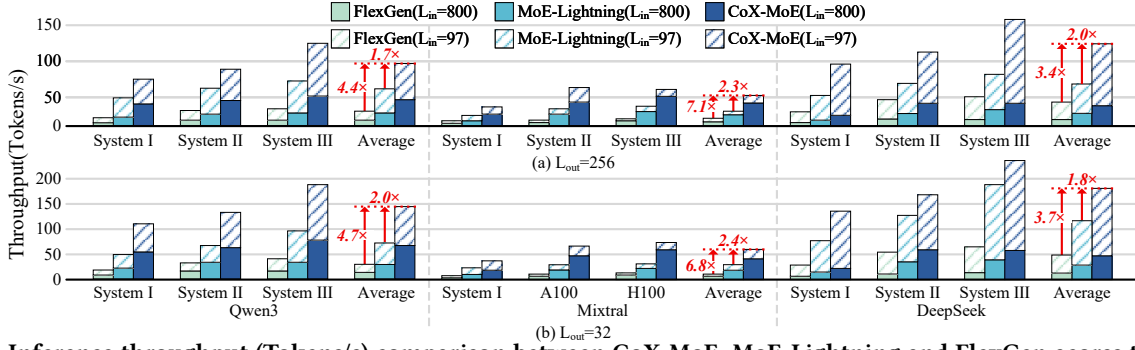


Figure 7: Inference throughput (Tokens/s) comparison between CoX-MoE, MoE-Lightning and FlexGen across the system configurations in Table. 2, with $B = 1024$

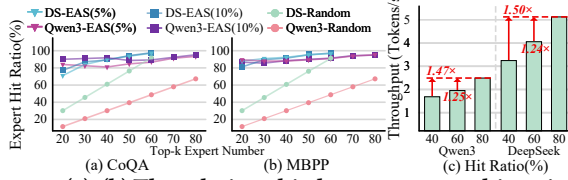


Figure 8: (a), (b) The relationship between expert hit ratio and number of stratified experts. (c) Evaluation of throughput (Tokens/s) versus hit ratio.

Table 3: Breakdown of proposed techniques. Baseline is MoE-Lightning with $B = 512$, $L_{in} = 1320$, $L_{out} = 128$, evaluated on System(I). (a) coalescing micro-batch and co-execution for experts with AMX, (b) attention offloading into CPU, (c) 80% expert hit ratio.

Technique	m	Throughput (Tokens/s)	Improvement
Baseline	256	16.8	–
+(a)	256	25.5	1.51×
+(a)+(b)	512	32.3	1.26×
+(a)+(b)+(c)	512	34.1	1.05×

Baseline. We evaluate CoX-MoE against two prominent open-source inference frameworks: FlexGen [22] and MoE-Lightning [2]. FlexGen introduces a basic offloading policy that searches the compute schedule for micro-batches and executes a zig-zag block schedule. MoE-Lightning is the SOTA batch inference system via CPU-GPU I/O (CGO) pipelining with paged expert weights and a hierarchical roofline model to select micro-batch. In both frameworks, attention in the decode stage is executed on the CPU.

6.2 Performance Comparison

Fig. 7 reports the throughput for batch size 1024 under two input lengths, ($L_{in} = 97$ and 800), and two output lengths, ($L_{out} = 32$ and 256). CoX-MoE demonstrates consistent improvements over the baselines across all L_{in} , L_{out} , models and systems. CoX-MoE delivers 1.7–2.4× and 3.4–7.1× higher throughput compared to MoE-Lightning and FlexGen, respectively. Whether the workload is prefill-dominant (driven by large L_{in}) or decode-dominant (driven by large L_{out}), CoX-MoE maintains a significant throughput advantage over both baselines and stages. This impact is driven by CoX-MoE’s coalescing-aware orchestration policy, which determines the strategies for compute and expert allocation to balance workloads and achieve efficient inference. In particular, the most significant improvements are observed in the Mixtral model, where

throughput increases by up to 2.4× over MoE-Lightning and 7.1× over FlexGen. Because Mixtral’s hidden dimension is roughly twice that of Qwen3 and DeepSeek, its decode-stage attention is more sensitive to transfer by PCIe. CoX-MoE’s orchestration policy, which strategically offloads OP_1 and OP_2 to the CPU, effectively mitigates the transfer via PCIe, leading to the substantial improvement.

6.3 Expert Hit Ratio Comparison

Figs. 8(a) and (b) show expert hit ratios of CoX-MoE’s EAS mechanism and the random selection for experts. In memory-constrained scenarios where VRAM can only hold a limited number of experts (e.g., up to 30 for DeepSeek and 50 for Qwen3), EAS achieves an expert hit ratio approximately 40% higher than random selection. As shown in Fig. 8(c), the ability to achieve a higher hit ratio directly translates to a throughput improvement of up to 1.47–1.50×.

6.4 Ablation Study

Table 3 shows how the components of our methods contribute to the performance. Performance was measured with Qwen3 on System (I). The most significant throughput improvement comes from coalescing expert micro-batches and performing co-execution with AMX, resulting in a 1.51× improvement. Furthermore, it is noteworthy that even method (c), which is limited to the prefill stage, provided a meaningful 1.05× improvement overall.

7 Conclusion

This paper presents CoX-MoE, an AMX-enabled CPU–GPU collaborative system that optimizes MoE inference through a coalesced micro-batch for expert execution with an offloading strategy to enhance throughput. By incorporating coalescing-aware orchestration policy and expert-aware stratification, CoX-MoE maximizes system utilization, achieving a 2.0× average higher throughput over SOTA methods. These results highlight CoX-MoE’s effectiveness in optimizing MoE inference for throughput-oriented workloads on resource-constrained single-GPU systems.

8 Acknowledgement

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) through the IITP-ITRC program (IITP-2025-RS-2020-II201847), and the IITP grant (No. RS-2025-02264029, Integration and Validation of an AI Semiconductor-Based Data Center Training and Inference System), all funded by the Korea government (MSIT).

References

- [1] Solution Brief. [n. d.]. Accelerate Artificial Intelligence (AI) Workloads with Intel Advanced Matrix Extensions (Intel AMX). ([n. d.]).
- [2] Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E Gonzalez, Matei Zaharia, and Ion Stoica. 2025. Moe-lightning: High-throughput moe inference on memory-constrained gpus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 715–730.
- [3] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.
- [4] Hongtao Chen, Weiye Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, et al. 2025. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 1014–1029.
- [5] Zixiang Chen, Yihe Deng, Yue Wu, Quanquan Gu, and Yuanzhi Li. 2022. Towards understanding mixture of experts in deep learning. *arXiv preprint arXiv:2208.02813* (2022).
- [6] Jack Choquette. 2023. Nvidia hopper h100 gpu: Scaling performance. *IEEE Micro* 43, 3 (2023), 9–17.
- [7] Jack Choquette and Wish Gandhi. 2020. Nvidia a100 gpu: Performance & innovation for gpu computing. In *2020 IEEE Hot Chips 32 Symposium (HCS)*. IEEE Computer Society, 1–43.
- [8] Seongmin Hong, Seungjae Moon, Junsoo Kim, Sungjae Lee, Minsub Kim, Dongsoo Lee, and Joo-Young Kim. 2022. Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 616–630.
- [9] Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. 2024. Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1018–1031.
- [10] Intel Corporation. 2025. Deep Learning with AVX512 and DL Boost. <https://www.intel.com/content/www/us/en/developer/articles/guide/deep-learning-with-avx512-and-dl-boost.html>. Accessed: 2025-11-18.
- [11] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [12] Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasicki. 2024. Fiddler: Cpu-gpu orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033* (2024).
- [13] Hyungyo Kim, Gaohan Ye, Nachuan Wang, Amir Yazdanbakhsh, and Nam Sung Kim. 2024. Exploiting intel advanced matrix extensions (AMX) for large language model inference. *IEEE Computer Architecture Letters* 23, 1 (2024), 117–120.
- [14] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. 2022. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110* (2022).
- [15] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [16] Jiacheng Liu, Peng Tang, Wenfeng Wang, Yuhang Ren, Xiaofeng Hou, Pheng-Ann Heng, Minyi Guo, and Chao Li. 2024. A survey on inference optimization techniques for mixture of experts models. *arXiv preprint arXiv:2412.14219* (2024).
- [17] Yingzhou Lu, Minjie Shen, Huazheng Wang, Xiao Wang, Capucine van Rechem, Tianfan Fu, and Wenqi Wei. 2023. Machine learning for synthetic data generation: a review. *arXiv preprint arXiv:2302.04062* (2023).
- [18] Shashi Narayan, Shay B Cohen, and Mirella Lapata. 2018. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745* (2018).
- [19] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2025. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology* 16, 5 (2025), 1–72.
- [20] NVIDIA. 2024. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>. Accessed: 2025-11-17.
- [21] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [22] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [23] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. Hobbit: A mixed precision expert offloading system for fast moe inference. *arXiv preprint arXiv:2411.01433* (2024).
- [24] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2024. Moe-infinity: Offloading-efficient moe model serving. *arXiv preprint arXiv:2401.14361* (2024).
- [25] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [26] Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient moe inference. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [27] Shuzhang Zhong, Yanfan Sun, Ling Liang, Runsheng Wang, Ru Huang, and Meng Li. 2025. HybriMoE: Hybrid CPU-GPU Scheduling and Cache Management for Efficient MoE Inference. *arXiv preprint arXiv:2504.05897* (2025).
- [28] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.